



南京大學

本科畢業論文

院 系 化学化工学院

专 业 化学

题 目 第一行标题

第二行标题

第三行标题

年 级 2018 学 号 181850195

学生姓名 姓名

指导教师 导师姓名 职 称 教授

提交日期 2022 年 5 月 20 日

南京大学本科生毕业论文（设计、作品）中文摘要

题目：第一行标题第二行标题第三行标题

院系：化学化工学院

专业：化学

本科生姓名：姓名

指导教师（姓名、职称）：导师姓名 教授

摘要：

哈希表是支持动态数据操作的基础数据结构，其时空权衡一直是算法研究的核心议题。2021 年，Bender 等人提出了一种基于 k -kick 树的新型哈希方案，从理论上证明可通过将更新操作的时间复杂度放宽至 $O(k)$ ，将每元素的空间浪费指数级压缩至 $O(\log^{(k)} n)$ 比特，打破了此前 $\Omega(\log \log n)$ 的空间下界。本文旨在复现该哈希算法，构建分层弹性哈希存储结构，实现 k -kick 机制的元素重定位逻辑与紧凑全局路由表，并将其封装为独立库集成到 LevelDB 数据库存储引擎中。实验结果表明，该方案在查询和删除操作上显著优于传统平衡树，同时在空间效率上展现出明显优势，验证了论文理论的工程可行性。

关键词：哈希表，时空权衡， k -kick 树，弹性哈希，LevelDB

关键词：我；就是；充数的；关键词

南京大学本科生毕业论文（设计、作品）英文摘要

THESIS: My Title in English

DEPARTMENT: School of Chemistry and Chemical Engineering

SPECIALIZATION: Chemistry

UNDERGRADUATE: Ming Xing

MENTOR: Professor My Supervisor

ABSTRACT:

Hash tables are fundamental data structures supporting dynamic data operations, with their time-space tradeoffs being a core topic in algorithm research. In 2021, Bender et al. proposed a novel hashing scheme based on k -kick trees, theoretically proving that by relaxing update time to $O(k)$, per-element space waste can be exponentially compressed to $O(\log^{(k)} n)$ bits, breaking the previous $\Omega(\log \log n)$ lower bound. This thesis implements the hashing algorithm, constructs a hierarchical elastic hash storage structure, implements the k -kick element relocation mechanism and a compact global routing table, and integrates it as an independent library into the LevelDB storage engine. Experimental results demonstrate significant query and deletion speed advantages over traditional balanced trees and notable space efficiency, validating the practical feasibility of the theoretical scheme.

Keywords: hash table, time-space tradeoff, k -kick tree, elastic hashing, LevelDB

KEYWORDS: Dummy; Keywords; Here; It Is

目 录

第一章 引言

1.1 研究背景和意义

哈希表是现代数据库管理系统、缓存系统和搜索引擎中最为基础的数据结构之一。其核心目标是支持动态插入和删除操作，并保证查询的时间复杂度为 $O(1)$ ，即在理想状态下，查询、插入、删除的操作所需时间为常数级，不随存储数据量 n 的增大而增加。为了实现这一点，哈希表往往需要在空间占用上做出让步，通过额外的比特位存储信息来实现极低的时间复杂度。

在要求插入、删除及查询操作均维持恒定时间复杂度（即期望时间或最坏情况时间为 $O(1)$ ）的前提下，早期的理论工作 [pagh2001low](#) 表明，在严格限制更新操作为 $O(1)$ 时间且仅使用简单探测序列的模型下，每个键值对的空间冗余往往难以低于 $\Omega(\log \log n)$ 比特。这一理论下界长期制约着哈希表空间效率的进一步提升。

2021 年, [Bender](#) 等人发表的论文《关于哈希表最优时间/空间权衡》[bender2022optimal](#) 打破了这一僵局。这项研究提出了一种基于 k -kick 树的新哈希方案，证明通过适当提高插入和删除操作的时间复杂度（从 $O(1)$ 提高到 $O(k)$ ，其中 k 是一个人为指定的具有一定上限的常数），空间浪费可以以指数形式降低至 $O(\log^{(k)} n)$ 比特。这里 $\log^{(k)} n$ 表示对 n 进行 k 次迭代对数运算，即 $\log^{(k)} n = \log(\log^{(k-1)} n)$ ，并定义 $\log^{(0)} n = n$ 、 $\log^{(1)} n = \log n$ 。当 $k = \log^* n$ （等式右侧表示对 n 进行迭代求对数直到结果小于等于 1 所需的次数，此时 $\log^{(k)} n = O(1)$ ）时，空间浪费可以降低到恒定的 $O(1)$ 比特水平，实现了理论和工程上的重大突破。

本毕业设计旨在复现这一前沿的哈希算法，并将其集成到常见的数据库管理系统中。研究意义如下：

理论价值：通过实验对论文理论进行验证，通过调整 k 的值研究 k 如何影响时间与空间复杂度，探索最优的 k 值选择策略。

工程价值：减少内存使用意味着成本降低和并发能力增强，在内存有限的场

景中提高现有数据库的存储效率。将其集成到数据库观察空间存储效率提升效果。

1.2 研究现状

哈希表作为支持几乎常数时间的插入、删除和查询操作的核心数据结构，其时间复杂度和空间复杂度之间的权衡一直是现代算法的研究焦点。

1.2.1 链式寻址法

链式寻址法通过链表处理冲突和碰撞的问题，它在实现上相对简单并且支持动态扩容，但每个节点都需要存储额外的指针开销，在查询时的探测深度也相当不平衡。链表节点在内存中的非连续分布还会破坏缓存局部性，导致更高概率的缓存缺失。

1.2.2 开放寻址法

开放寻址法（如线性探测、二次探测）将所有元素存储于连续数组中，通过一个固定的探测序列函数处理冲突。当当前元素哈希结果与已存入的元素冲突时，沿着探测序列寻找下一个位置，直到不发生冲突。然而，随着负载因子 α 趋近于 1，冲突链长度急剧增加，导致查询与插入因为要沿着长长的探测序列进行操作，时间复杂度退化。因此其难以在保证 $O(1)$ 时间复杂度的同时维持极高的空间利用率。

1.2.3 Cuckoo Hashing

Cuckoo Hashing 由 Pagh 和 Rodler 于 2001 年提出^{pagh2004cuckoo}，通过引入多候选位置及元素重定位机制，在最坏情况下实现了 $O(1)$ 的查询时间。多候选位置改变了先前使用一个关于探测次数的探测函数生成探测序列的方式，转而采用多个哈希函数为每个元素生成多个可存入位置，每一个元素只会出现在属于自己的可存入位置中。元素重定位机制则从发现冲突退让并探测转变为发现可存入位置都有冲突就踢出一个元素，将该元素往属于它的其他可存入位置移动，

重复进行直到不再冲突。这两种机制协作严格保证了任何键的查找操作只需要计算预先设置的哈希函数个数的次数，实现了常数级查询。

1.2.4 基于 k -kick 树的时空最优方案

上述所有解决方案均未能突破前面提出的空间限制下界。而 2021 年发表的论文《关于哈希表时间/空间权衡的最优方案》^{bender2022optimal}中提出的利用 k -kick 树结构构建哈希表方案是目前最新的突破成果。该方案将哈希表映射问题转化为一个层次化的“球到槽”问题，通过引入随机优先级和有限的不超过人为确定的 k 次元素移动（即“ k -kick”），从理论上证明，对于任何 $k \geq 1$ ，都可以实现 $O(k)$ 时间复杂度的哈希表元素更新（插入、删除、查询）和 $O(\log^{(k)} n)$ 比特的空间浪费。

该研究还证明了此权衡曲线对于所有基于“增强开放寻址”框架的哈希表是紧确的^{bender2022optimal}，即任何此类方案若更新时间为 t ，则每元素必须使用 $\Omega(\log^{(t)} n)$ 位的冗余。这意味着，若要在次对数级的更新时间下实现 $O(1)$ 位的空间冗余，必须彻底脱离“增强开放寻址”的范式。

1.3 主要研究内容和目标

本选题旨在复现并验证 2021 年提出的基于 k -kick 树的最优时空权衡哈希算法，并探索其在数据库存储中的实际应用潜力。具体研究内容包括以下四个目标：

1. **数据结构的实现和算法复现：**依照论文中的定义，构建基于 k -kick 树的分层存储结构，并实现从根节点到叶节点的动态元素分配逻辑。实现高效的 kick 策略，模拟论文中的随机优先级机制，建立在插入冲突时通过 k 次元素移动将新元素安置或触发上层重组并完成的机制，保证 $O(k)$ 的时间复杂度。实现紧凑的辅助路由结构，用于快速定位元素所在的探测层级。
2. **数据库存储集成与适配：**设计形式化的哈希表操作接口，对外只暴露 insert、delete、get 等操作，屏蔽底层逻辑设计提高兼容性。在技术和时间允许的情况下，进阶解决多线程环境下的锁竞争问题，可尝试探索如何将存储树持久化到磁盘。

3. **理论正确性验证与复杂度分析:** 基于代码实现, 形式化验证算法复杂度, 结合理论推导与实验数据, 量化分析参数 k 对更新时间 $T(n)$ 和空间浪费 $S(n)$ 的具体影响, 验证 $S(n) = O(\log^{(k)} n)$ 的正确性。
4. **性能对比实验:** 构建包含链式哈希、线性探测、Cuckoo Hashing 等主流算法以及标准库中的 `std::unordered_map` 和 `std::map` 作为对照组, 在相同硬件环境下进行全方位性能比对。测试不同数据规模下插入、查询、删除操作的时间开销和空间占用情况。

1.4 论文结构

本文共分为六章。第一章为引言, 介绍研究背景、现状、目标与论文结构。第二章介绍相关理论基础, 包括哈希表的基本概念、Cuckoo Hashing 原理及 k -kick 树哈希方案的详细理论。第三章详细描述系统的设计方案, 包括分层存储结构、kick 重定位机制、紧凑路由表以及位置编码压缩存储的设计。第四章介绍算法实现细节, 包括核心数据结构的 C++ 实现、关键算法流程以及数据库集成的接口设计。第五章展示实验结果与分析, 对多种数据规模下的插入、查询、删除性能和空间占用进行对比评估。第六章总结全文工作, 分析当前方案的优缺点并展望未来改进方向。

致 谢

感谢论文作者 Bender 等人提出的创新性哈希方案，为本毕业设计提供了理论基础。感谢南京大学 Linux 用户组提供的NJUThesis 毕业论文模板。

